

F L U T T E R B U I L D

flutter_build 学习手册

在 Linux 上交叉编译 Flutter Windows 桌面应用

LLVM-MinGW + Wine 全开源工具链深度剖析 · 面向学习

flutter_build 学习手册

在 Linux 上交叉编译 Flutter Windows 桌面应用 —— LLVM-MinGW + Wine 全开源工具链深度剖析

项目	说明
适用版本	flutter_build 0.1.0-dev
目标平台	Linux x86_64 (推荐 Ubuntu 24.04) → Windows x86_64
语言	Dart (核心), 涉及 C/C++, CMake, Shell
许可	Apache-2.0
本手册	面向学习, 系统讲解项目定位、原理、源码实现与运维

如何使用本手册

本手册按"由宏观到微观、由原理到实现"的顺序组织, 共五部分二十一章加附录, 排版目标为 A4 打印约 20 页。建议阅读路径:

- 想快速了解它能做什么: 读第 1、2、3 章 (概览与原理)。
- 想动手用起来: 直接跳第 18、19 章 (命令与部署) 与附录 A、B、C (速查与排错)。
- 想读懂源码实现: 第三、四部分是核心, 逐阶段拆解 lib/src/build/ 下的流水线代码。
- 做二次开发或面试准备: 通读全书, 重点体会第五部分的基础设施设计与各处的"版本接缝"注释。

每章开头给出「对应源码」定位, 正文用「问题 → 思路 → 实现」的结构展开, 关键代码片段后附中文注释。

目录

第一部分·概览与原理 - 第 1 章 项目定位与要解决的问题 - 第 2 章 交叉编译的整体架构与五步流水线 - 第 3 章 七条关键技术决策

第二部分·工具链与环境探测 - 第 4 章 Flutter SDK 探测: FlutterEnv - 第 5 章 目标工程解析: FlutterProject - 第 6 章 交叉工具链供给: Toolchain 与 LLVM-MinGW - 第 7 章 引擎产物供给: EngineArtifacts 与版本接缝 - 第 8 章 本地缓存布局: CachePaths

第三部分·构建流水线逐阶段精讲 - 第 9 章 流水线编排、并行调度与 BuildContext - 第 10 章 阶段 1: 源码暂存与 ephemeral 生成 - 第 11 章 阶段 2: MSVC 编译标志翻译 - 第 12 章 阶段 3: Dart Kernel 编译 - 第 13 章 阶段 4: AOT 编译 (Wine + gen_snapshot) - 第 14 章 阶段 5: CMake 配置与构建 - 第 15 章 阶段 6 资源打包与阶段 7 产物组装

第四部分·兼容性工程 - 第 16 章 MinGW 兼容垫片与插件源码补丁 - 第 17 章 增量构建机制 - 第 18 章 调试仪表: 把引擎日志接回控制台

第五部分·CLI、部署与基础设施 - 第 19 章 命令详解: doctor / precache / windows / clean - 第 20 章 远程部署: config.yaml 与 scp - 第 21 章 基础设施: 日志 / 进程 / 异常 / 文件系统

附录 - 附录 A 命令与标志速查表 - 附录 B 环境变量速查表 - 附录 C 故障排查清单 - 附录 D 术语表 - 附录 E 源码文件地图

第一部分 · 概览与原理

第 1 章 项目定位与要解决的问题

Flutter 官方只能在 Windows 主机上用 MSVC + Windows SDK 构建 `flutter build windows`。这给团队带来几道现实的墙：

- 1. 许可证与平台锁定：CI 机器、开发者笔记本往往只有 Linux；为编译 Windows 版本专门维护 Windows 虚拟机/机器成本高。
- 2. 专有工具链：MSVC 不可自由分发，难以容器化。
- 3. 产物一致性：任何替代方案产出的 `.exe` 必须能在真实 Windows 上运行，且与官方产物行为一致。

`flutter_build` 的答案是：完全用开源工具链，在 Linux 上交叉编译出与官方字节级兼容的 Windows 桌面应用。它不动你的 Flutter 工程源码，只是把"构建"这件事换了一套底层工具：

环节	官方 Windows 做法	flutter_build 的 Linux 替代
C/C++ 编译	MSVC <code>cl.exe</code>	LLVM-MinGW 的 <code>clang</code> / <code>clang++</code>
链接	MSVC <code>link.exe</code>	LLD (<code>ld.lld</code>)
Windows SDK 头/库	微软 SDK	mingw-w64 头与 <code>.a</code> 导入库
Dart → 机器码 AOT	<code>gen_snapshot.exe</code> 原生运行	同一个 <code>gen_snapshot.exe</code> ，在 Wine 下运行
构建系统	CMake + Ninja	同一套 CMake + Ninja
引擎 DLL	<code>flutter_windows.dll</code>	原样复用官方产物（不改一个字节）

核心思想一句话：用官方 Windows 引擎产物 + Wine 跑官方 AOT 编译器 + MinGW 编译并链接原生代码，从而绕过对 MSVC 与 Windows 的依赖。

产物形态与官方完全一致：

```
<app>/
├─ <app>.exe          # 你的应用主程序
├─ flutter_windows.dll # Flutter 引擎（原样复用）
└─ data/
    ├─ icudtl.dat      # ICU 国际化数据
    ├─ app.so           # AOT 机器码快照 (release/profile)
    └─ flutter_assets/  # 图片/字体/AssetManifest 等资源
```

设计红线（贯穿全书）：能不碰源码就不碰源码。所有兼容性问题优先在 `flutter_build` 这一侧用"编译标志 / 垫片头文件 / CMake 配置"解决；实在躲不掉的才在暂存副本上打源码补丁，且补丁必须保持 MSVC 可原样构建——绝不去修改 pub-cache 原件或用户工程。

第 2 章 交叉编译的整体架构与五步流水线

一次 `flutter_build windows` 背后是七个阶段（Stage）组成的流水线。README 用五步概括其原理，源码里则拆成七个可独立测试的阶段——多出的第 6 阶段（资源打包）独立成轨，以便与 CMake 构建重叠执行。七个阶段按三条并行 lane 分组如下：

轨道 (lane)	阶段	做什么	产物
原生轨 native	阶段 1 暂存	复制 <code>windows/</code> ，生成 <code>ephemeral/</code> 、插件符号链接	—
	阶段 2 翻译	把 CMakeLists 里的 MSVC 标志翻成 Clang 等价	—
	阶段 5 CMake	LLVM-MinGW <code>clang</code> / <code>lld</code> 编译链接 runner + 插件	<code><app>.exe</code> (经 C ABI 链接 <code>flutter_windows.dll</code>)
Dart 轨 dart	阶段 3 kernel	<code>frontend_server.dart.snapshot</code> (host Dart VM)	<code>app.dill</code> (平台无关的 Dart kernel 快照)
	阶段 4 AOT	Wine + <code>gen_snapshot.exe</code> (Windows PE 二进制)	<code>app.so</code> (ELF 容器，内含 x86_64 机器码)
资源轨 assets	阶段 6 资源	<code>copy_flutter_bundle</code> (门控：等 kernel 完成后再启动)	<code>flutter_assets/</code> (纯 host Dart，与 AOT、CMake 并行)
汇合 <code>join</code>	阶段 7 组装	产物组装成最终可分发包 (串行汇合点)	最终可分发包

三 lane 并行：原生轨（暂存→翻译→CMake）、Dart 轨（kernel→AOT）、资源轨（copy_flutter_bundle）三条 lane 的产物文件集互不重叠，可安全并行。资源轨是纯 host Dart 任务，为避免与 kernel 编译两个 Dart 进程争用工程 `.dart_tool/`，它带一道跨轨门控——等 kernel 阶段结束再启动，随后仍与 AOT、CMake 重叠。全部 lane 完成后，串行执行汇合的组装阶段（阶段 7）。调度决策是纯函数（`planSchedule`），可用 `--no-parallel` 回退为原始串行次序；每次构建结束打印逐阶段计时报告，并行模式下还给出相对串行的节省比例（详见第 9 章）。

为什么是这条路？Dart 应用的编译天然分成两段：

- 前端（平台无关）：`frontend_server` 把 Dart 源码 + Flutter 框架编译成 **kernel**（`.dill`，一种中间字节码）。这一步在 host 的 Dart VM 上跑，与目标平台无关。
- 后端（平台相关）：`gen_snapshot` 把 kernel 编译成目标平台的 **AOT 机器码**。Flutter Windows 引擎在运行时以 **ELF 加载器**消费 `app.so`——而 `gen_snapshot --snapshot-kind=app-aot-elf` 无论宿主是什么系统都产出 ELF，于是“在 Linux 上、用 Windows 版编译器、产出让 Windows 引擎消费的 ELF”这条看似矛盾的路径恰好成立。

原生代码侧（runner 窗口程序 + 各插件的 C++）则由 CMake 驱动、用 MinGW 编译，最终以 C ABI 与官方 `flutter_windows.dll` 链接。

debug / profile / release 三种模式的差异（`WindowsFlavor`）：

模式	是否 AOT	用哪个引擎 DLL	是否跑阶段 4	Dart VM 形态
debug	否 (JIT)	<code>windows-x64</code> (JIT 引擎)	跳过	带 kernel_blob，可 hot reload
profile	是	<code>windows-x64-profile</code>	运行	AOT + observatory
release	是	<code>windows-x64-release</code>	运行	AOT, product VM

流水线里 `AotCompileStage.shouldRun` 直接返回 `ctx.mode.isAot`，因此 debug 构建根本没有阶段 4，阶段总数动态变为 6。

第 3 章 七条关键技术决策

README「Key design decisions」列了七条，它们正是理解全项目的钥匙。逐条结合源码展开：

决策 1：flutter_windows.dll 原样复用

它导出的是一套**纯 C ABI**。C ABI 在各编译器间稳定，因此 MSVC 构建的 DLL 能与 MinGW 编译的目标文件干净链接。MinGW 链接所需的导入库要么复用引擎缓存里的 `flutter_windows.dll.lib`，要么用 `llvm-dlltool` 现场重新生成。阶段 1 会把这个 `.dll` 与 `.lib` 铺进 `ephemeral/`。

决策 2：重写 `windows/flutter/CMakeLists.txt`

Flutter 原版脚手架里有一段"Flutter tool backend", 会在构建时回调 `flutter assemble` ——那会在 host 上重新进入 Dart 工具链, Linux 交叉场景下必然失败。`neutralizeFlutterAssemble()` 找到 `# === Flutter tool backend` `===` 标记后截断其内容, 替换为一个空的 `flutter_assemble` 目标 (保留它只为满足其它目标的 `add_dependencies`)。

```
// flutter_ephemeral.dart
const String kFlutterToolBackendMarker = '# === Flutter tool backend ===';
String neutralizeFlutterAssemble(String cmakeContent) {
  final idx = cmakeContent.indexOf(kFlutterToolBackendMarker);
  if (idx < 0) return cmakeContent;      // 找不到标记就原样返回
  final head = cmakeContent.substring(0, idx);
  return '$head'
    '# === Flutter tool backend (由 flutter_build 中和) ===\n'
    'add_custom_target(flutter_assemble)\n'; // 空目标占位
}
```

决策 3：MSVC 标志翻译

插件与脚手架的 CMakeLists 常带 MSVC 专属写法 (`/W3`、`/EHsc`、`/std:c++17`、`/utf-8`、`"xxx.lib"`)。`MsvcFlagTranslator` 用正则把它们重写成 Clang 等价物, 且只作用于暂存副本。详见第 11 章。

决策 4：静态链接 C++ / pthread 运行时

`-static-libstdc++ -static-libgcc -static-libwinpthread` (在 CMake 阶段以 `-static` 统一施加) 使产物不依赖 MinGW 的运行时 DLL (`libc++.dll`、`libunwind.dll` 这些普通 Windows 上没有的文件), 实现自包含分发。

决策 5：ELF AOT 快照

`gen_snapshot --snapshot-kind=app-aot-elf` 无论目标 OS 都写 ELF 文件; Flutter Windows 引擎内建 ELF 加载器, 运行时直接消费它。这条与决策 1、第 2 章的后端思路呼应。

决策 6：尽量少改被编译的应用

修复交叉编译问题时, 优先把方案留在 flutter_build 一侧 (编译标志 `-Wno-...`、MinGW 垫片头、CMake 配置), 源码级补丁是最后手段, 只打在物化副本上, 且保持 MSVC 兼容。这条是前五条背后的"宪法"。

决策 7：三 lane 并行 + 保守跨轨门控

原生轨 (暂存→翻译→CMake)、Dart 轨 (kernel→AOT)、资源轨 (`copy_flutter_bundle`) 三条 lane 的产物文件集互不重叠, 并行无竞争; 唯一的隐性冲突是资源轨的 `flutter assemble` 与 kernel 的 frontend_server 同为 host Dart 进程, 可能争用工程 `.dart_tool/`。因此资源轨带一道保守门控: 等 kernel 阶段结束 (无论成功/跳过/失败, 均在 `finally` 里放行 Completer) 再启动, 随后仍与 AOT、CMake 重叠。调度被抽成纯函数 `planSchedule`, 可脱离真实构建单测; `--no-parallel` 一键回退串行。详见第 9 章。

第二部分 · 工具链与环境探测

本部分讲"开工前把哪些东西准备好、放在哪"。CLI 入口在构建前依次解析四份"探测结果", 它们最终都汇入第三部分的 `BuildContext`: `FlutterEnv` (SDK)、`FlutterProject` (工程)、`Toolchain` (交叉工具链)、`EngineArtifacts` (引擎产物), 外加 `CachePaths` (缓存目录)。

第 4 章 Flutter SDK 探测: `FlutterEnv`

对应源码: `lib/src/flutter_env.dart`

要解决的问题: 找到用户装的 Flutter SDK, 并定位构建所需的内部文件 (Dart 可执行、frontend_server 快照、patched_sdk), 且不能每次都 `flutter --version` (冷启动约 2 秒)。

思路: 在 PATH 上 `which flutter` → 解析符号链接 (兼容 asdf / homebrew / snap) → 向上两级得到 `sdkRoot` → 直接读文件布局。

```
final flutter = flutterExecutable ?? await r.which('flutter');
final resolved = await File(flutter).resolveSymbolicLinks();
final sdkRoot = p.normalize(p.join(p.dirname(resolved), '..')); // <sdk>/bin/flutter → <sdk>
```

产出的关键字段: `sdkRoot`、`flutterVersion`、`dartSdkVersion`、`engineCommitHash` (来自 `bin/internal/engine.version`)、`storageBaseUrl` (尊重 `FLUTTER_STORAGE_BASE_URL`)、`dartExecutable`、`frontendServerSnapshot`、`hostEngineDir`。

版本接缝 (本文件是升级 Flutter 的哨兵)

Flutter 目录布局随版本变过几次, 代码集中处理:

- 顶层 `version` 文件在约 3.13+ 被移除 (改由 git 计算) → 视为可选, 缺失时回退到 `FLUTTER_VERSION` 环境变量或 `"unknown"`; 该值仅用于诊断, 从不参与构建正确性判断。
- `frontend_server` 快照在约 3.16+ 更名为 `frontend_server_aot.dart.snapshot` 且可能位于 `dart-sdk/bin/snapshots/` → 用候选列表依次探测:

```
final frontendServerCandidates = <String>[
  p.join(hostEngineDir, 'frontend_server_aot.dart.snapshot'),
  p.join(hostEngineDir, 'frontend_server.dart.snapshot'),
  p.join(sdkRoot, 'bin/cache/dart-sdk/bin/snapshots', 'frontend_server_aot.dart.snapshot'),
  p.join(sdkRoot, 'bin/cache/dart-sdk/bin/snapshots', 'frontend_server.dart.snapshot'),
];
```

AOT 快照必须用 `dartaotruntime` 跑

现代 `frontend_server_aot.dart.snapshot` 是 AOT 快照, 用普通 `dart` 运行会报 `is an AOT snapshot ... 'dartaotruntime'` (exit 255)。因此:

```
bool get frontendServerIsAot => p.basename(frontendServerSnapshot).contains('_aot');
String get frontendServerRuntime =>
  frontendServerIsAot ? dartAotRuntimeExecutable : dartExecutable;
```

`locate()` 里还会提前校验 `dartaotruntime` 是否存在, 把错误暴露在探测阶段而非耗时的阶段 3。`patchedSdkPath(product:)` 依据是否 `product` 模式返回 `flutter_patched_sdk` 或 `flutter_patched_sdk_product`。

第 5 章 目标工程解析: FlutterProject

对应源码: `lib/src/project.dart`

职责: 把"当前目录"读成一份构建契约——应用元数据 + Windows 插件清单。工具不主动跑 `flutter pub get`, 而是要求调用方先解析好, 并检查 `.dart_tool/package_config.json` 是否存在。

`load()` 的校验链: 有 `pubspec.yaml` → 解析为 YAML map → 含 `flutter:` 段 → 有 `lib/main.dart` 入口 → 有 `package_config.json` (否则抛 `ProjectException` 并提示跑 `flutter pub get`)。

Windows 插件解析

`_resolveWindowsPlugins` 遍历 `package_config.json` 里每个包的 `pubspec.yaml`, 看 `flutter.plugin.platforms.windows` 是否存在, 据此构造 `WindowsPluginRef`:

```
class WindowsPluginRef {
  final String name;          // 如 url_launcher_windows
  final String rootPath;      // 包根目录
  final String pluginClass;    // C++ 注册类名
  String get windowsCMakeDir => p.join(rootPath, 'windows');
  bool get hasNativeCode =>
    File(p.join(windowsCMakeDir, 'CMakeLists.txt')).existsSync();
}
```

`hasNativeCode` 区分"有原生 C++ (要参与 CMake 构建)"与"纯 Dart 插件", 后面的符号链接、源码补丁、DLL 打包都以此为准。

生成的三类文件

`FlutterProject` 还负责渲染与官方一致的构建输入: `renderGeneratedPluginsCmake()` (插件清单, 含关键的 `set_target_properties(... PREFIX "" IMPORT_PREFIX "")`, 见第 10 章)、`renderGeneratedPluginRegistrant()` (C++ 侧注册代码)。此外暴露两个路径 getter: `packageConfig`、`dartPluginRegistrant` (后者是纯 Dart 插件的注册器, 阶段 3 要用它避免运行时 `MissingPluginException`)。

第 6 章 交叉工具链供给: Toolchain 与 LLVM-MinGW

对应源码: `lib/src/toolchain.dart`

`Toolchain` 是一份"已解析好的可执行文件路径表", `ToolchainProvisioner` 则负责把它凑齐。

两种后端

```
enum ToolchainBackend { llvmMingw, systemMingw }
```

- **llvmMingw (推荐)**: clang + lld + mingw-w64, 路径形如 `<root>/bin/x86_64-w64-mingw32-clang`。
- **systemMingw (后备)**: apt 装的 GCC-MinGW, 二进制在 PATH 上, 用 `/usr` 作象征性根。

`Toolchain` 的每个 getter (`clang`/`clangxx`/`windres`/`lldLink`/`llvmDlTool`/`llvmAr`/`llvmRanlib`) 都按后端切换路径。有个易被忽略但关键的成员:

```
// clang 驱动内置了 sysroot 头搜索路径, 但资源编译器 llvm-rc 没有—
// 编译 .rc 时必须显式 -I 传入, 否则报 'winres.h' file not found.
String get mingwSysrootInclude => p.join(llvmMingwRoot, targetTriple, 'include');
```

LLVM-MinGW 版本固定

`LlvmMingwRelease` 描述一个 release；默认固定 `20260922` + `ucrt`，并带 sha256 校验。为什么要固定？避免版本漂移导致不可再现构建。

```
final defaultLlvmMingw = LlvmMingwRelease.pinned(
  version: '20260922', crt: 'ucrt',
  sha256: 'bb7bb7654b33d5aa8712acb837c963b2e0c56352560c76105270a3268c665c21',
);
```

Linux distro tag 的玄机：llvm-mingw 每个 release 只发一个 Linux x86_64 构建，故意选较老的 Ubuntu 编译，使其 glibc 够老从而**向前兼容**更新的发行版（新系统能跑旧二进制，反之不行）。所以 `ubuntu-20.04` 构建跑在 24.04 上完全正常。tag 由 `_linuxTagByVersion` 表按版本解析，升级版本时登记对应 tag 即可，避免下载 404。

四级供给优先级

`provision()` 从高到低：

1. 用户显式路径：`--toolchain-path` 或环境变量 `LLVM_MINGW_ROOT` → 校验 `<root>/bin/<triple>-clang` 存在后直接接用，不下载。
2. 缓存命中：`~/.flutter_build/toolchains/<name>/.installed` 标记存在 → 复用。
3. 自动下载：支持 `FLUTTER_BUILD_MIRROR` 镜像；下载到 `downloads/`，可选 sha256 校验，用 `tar -xJf` 流式解压（避免把 ~250MB 全 load 进内存），成功后写 `.installed` 标记。
4. 系统 apt GCC-MinGW 后备：`which x86_64-w64-mingw32-gcc` 命中则用 `systemMingw`，否则抛带三条建议的 `ToolException`。

`allowDownload=false` (`--no-precache`) 时缺工具链直接抛错而非下载。下载实现里有个细节：已存在且非零大小的压缩包直接复用，用 `.part` 临时文件 + rename 保证原子性。

第 7 章 引擎产物供给：`EngineArtifacts` 与版本接缝

对应源码：`lib/src/engine_artifacts.dart`

策略：不自己重造 URL/产物方案（Flutter 的产物命名换过好几种），而是委托 `flutter precache --windows`，再用类型化 getter 暴露具体文件路径。

产物目录布局 (`_resolve()`)：

```
<flutter>/bin/cache/artifacts/engine/
├─ windows-x64/           # embedderDir: flutter_windows.dll、头文件、cpp_client_wrapper、.lib、(旧版)icudtl
├─ windows-x64-release/   # releaseArtifactsDir: gen_snapshot.exe (AOT)
├─ windows-x64-profile/   # profileArtifactsDir: gen_snapshot.exe
└─ linux-x64/             # hostEngineDir: 新版共享的 icudtl.dat
```

按模式选对引擎 DLL（一个致命坑）

```
String flutterWindowsDllForMode(WindowsFlavor mode) {
  final dir = switch (mode) {
    WindowsFlavor.release => releaseArtifactsDir, // AOT 引擎
    WindowsFlavor.profile  => profileArtifactsDir, // AOT 引擎
    WindowsFlavor.debug    => embedderDir,         // JIT 引擎
  };
  return p.join(dir, 'flutter_windows.dll');
}
```

若 release 却用了 `windows-x64` 的 JIT 引擎，bundle 里只有 `app.so` (AOT)，启动即失败: *"Not running in AOT mode but could not resolve the kernel binary"*。

`icudtl` getter 也做了兼容：优先每平台副本（旧版），回退到共享的 `linux-x64/` 副本（新版）。

新鲜度判断 `isStale()` (2026-09 实测坑)

SDK 自动升级 (snap / `flutter upgrade`) 只刷新 host 侧的 dart-sdk / frontend_server (kernel 生产者)，而 Windows 引擎产物里的 `gen_snapshot.exe` (kernel 消费者) 残留旧版；同时 flutter 的 cache stamp 在 Linux 上会被"空更新"污染，普通 precache 静默跳过、无法自愈。错配特征：

```
Can't load Kernel binary: Invalid kernel binary format version (expected 130, found 138).
```

判据巧妙利用文件 mtime：产物 mtime 保留自 zip 内时间戳（引擎构建时间），同一引擎构建的 frontend_server 与 gen_snapshot 时间差在小时级，**差值超过 7 天**即判为来自不同引擎版本 → stale。frontend_server 缺失时保守放行不阻塞。

`ensure()` 的修复逻辑：stale 时**必须** `--force` 重新 precache；missing 场景普通 precache 也可能被 stamp 污染静默无效，故兜底再强刷一次；仍不行才抛出带 `describe()` 明细的 `ArtifactException`。

第 8 章 本地缓存布局: `CachePaths`

对应源码: `lib/src/cache_paths.dart`

集中管理工具自己的缓存，解析优先级（高到低）：`--cache-dir` → 环境变量 `FLUTTER_BUILD_CACHE` → `$XDG_CACHE_HOME/flutter_build` → `$HOME/.flutter_build`。

```
<root>/
├─ toolchains/llvm-mingw-<version>/ # 解压后的 LLVM-MinGW
├─ engine/<engine-hash>/{windows-x64,-release,-profile}
└─ downloads/ # 下载临时 zip/tar
```

注意区分两套缓存：`CachePaths` 是**工具级**（跨项目共享的工具链），而 `build/win_cross/` 是**项目级**（每次构建的中间与产物）。`clean` 只删后者，前者要手动 `rm -rf ~/.flutter_build`。

第三部分 · 构建流水线逐阶段精讲

本部分是全书核心，逐阶段拆解 `lib/src/build/` 下的实现。先讲编排器、并行调度与共享上下文，再按执行顺序精讲七个阶段。

第 9 章 流水线编排、并行调度与 `BuildContext`

对应源码：`lib/src/build/pipeline.dart`、`build_schedule.dart`、`stage_timing.dart`、`build_context.dart`、`stages/build_stage.dart`

`BuildContext`：不可变的"构建契约"

`BuildContext` 把四份探测结果（`env`/`project`/`artifacts`/`toolchain`）连同用户选项（`mode`、`dart-defines`、混淆、增量、`debug-console` 等）汇总，并据此派生出全流程约定的目录/文件路径，供各阶段共享，避免到处重复拼路径：

```
String get modeDir          => p.join(buildRoot, mode.cliName);           // ../release
String get windowsStageDir => p.join(modeDir, 'windows_src');           // 暂存的 CMake 源
String get cmakeBuildDir   => p.join(modeDir, 'cmake_build');           // CMake 配置/构建输出
String get intermediatesDir => p.join(modeDir, 'intermediates');         // kernel dill / AOT elf
String get kernelDill      => p.join(intermediatesDir, 'app.dill');
String get appAotElf       => p.join(intermediatesDir, 'app.so');
String get finalExe        => p.join(modeDir, appName, '$appName.exe');
String get outputDir       => p.dirname(finalExe);                       // bundle 目录
String get dataDir         => p.join(outputDir, 'data');
String get mingwCompatDir  => p.join(intermediatesDir, 'mingw_compat'); // 兼容垫片头目录
```

两个值得记住的选项：`incremental`（默认开，`kernel/AOT` 输入未变则跳过重编）、`dllSearchRoot`（预构建 `DLL` 搜索根，默认项目根祖父目录，收窄可加速大型工作区）。第三个开关 `parallel`（默认开，`--no-parallel` 关闭）决定流水线按三 lane 并行还是退回原始串行次序。

`BuildStage`：阶段抽象

每个阶段实现统一接口，可单独理解、单独测试：

```
abstract class BuildStage {
  String get name; // 日志分组标题，如 'compile Dart kernel'
  bool shouldRun(BuildContext ctx) => true; // AOT 阶段覆写为 ctx.mode.isAot
  Future<void> run(BuildContext ctx);
}
```

BuildPipeline: 编排器 + lane 执行

```
List<BuildStage> _stages() => [
  SourceStagingStage(),      // 阶段1 暂存 CMake 源
  TranslateFlagsStage(),     // 阶段2 翻译 MSVC 标志
  CompileKernelStage(),      // 阶段3 编译 kernel → app.dill
  AotCompileStage(),         // 阶段4 AOT → app.so (仅 release/profile)
  CMakeBuildStage(),         // 阶段5 CMake/Ninja → .exe
  FlutterAssetsStage(),      // 阶段6 资源打包 → flutter_assets/
  AssembleBundleStage(),     // 阶段7 组装 bundle
];

Future<void> run(BuildContext ctx) async {
  // 并行前的公共准备:
  // 1. intermediates 目录原由暂存阶段创建, 但并行时 kernel 可能先于暂存
  //    启动, 故提前创建。
  // 2. wine 包装脚本被 AOT 与 CMake 阶段共用, 提前一次原子落盘, 避免并行竞争。
  await Directory(ctx.intermediatesDir).create(recursive: true);
  await WineWrapper(toolchain: ctx.toolchain, buildRoot: ctx.buildRoot).materialize();

  final schedule = planSchedule(_stages(), parallel: ctx.parallel);

  // 为每个阶段建一个 Completer (按阶段名); 带 gate 的 lane 启动前 await
  // 前驱阶段的 completer。
  final gates = <String, Completer<void>>{ /* ... */ };
  final timings = <StageTiming>[];
  final wall = Stopwatch()..start();

  if (schedule.concurrent.isNotEmpty) {
    // Future.wait 默认 eagerError=false: 等所有 lane 收尾后再报首个错误,
    // 不会留下半死的子进程。资源轨会先等 kernel (gate)。
    await Future.wait(
      schedule.concurrent.map((l) => _runLane(l, ctx, timings, gates)));
  }
  for (final lane in schedule.serial) { // 汇合: 依赖并行组全部产物
    await _runLane(lane, ctx, timings, gates);
  }

  _log.success('Windows build complete: ${ctx.finalExe}');
  _reportTimings(timings, wall.elapsed, ctx.parallel); // 计时报告
}
```

关键点: 阶段清单面向 **Windows 目标固定** (7 项, 顺序是 `planSchedule` 的硬约定, 不符即抛 `ArgumentError`, 避免静默错位分组), 不是跨平台抽象层; `shouldRun` 为 false 的阶段 (如 debug 无 AOT) 被跳过, 但**不改变** lane 结构与门控关系。

planSchedule: 把 7 阶段切成并行 lane (`build_schedule.dart`)

调度决策被抽成**纯函数**, 可脱离真实构建单独单测分组、顺序与门控:

```

class StageLane {
  final String label;          // 日志分组前缀: native / dart / assets / join
  final List<BuildStage> stages; // lane 内按序串行
  final String? gateOnStageName; // 跨 lane 软依赖: 等其它 lane 的同名阶段完成
}

BuildSchedule planSchedule(List<BuildStage> stages, {required bool parallel}) {
  if (parallel) {
    return BuildSchedule(
      concurrent: [
        StageLane('native', [staging, translate, cmake]),
        StageLane('dart', [kernel, aot]),
        // 保守门控: 资源轨等 kernel 完成再启动 (避开两个 host Dart 进程
        // 争用 .dart_tool), 启动后仍与 AOT、原生轨 CMake 并行。
        StageLane('assets', [assets], gateOnStageName: kernel.name),
      ],
      serial: [StageLane('join', [assemble])],
    );
  }
  // 顺序回退: concurrent 置空, 全部按原始次序落 serial 依序执行。
}

```

为什么这样切是安全的？ 三条前置 lane 的产物文件集互不重叠：原生轨只写 `windows_src/`、`cmake_build/` 与 `exe`；Dart 轨只写 `intermediates/` 下的 `dill/elf`；资源轨只写 `flutter_assets/`。唯一的隐性冲突在 host Dart 侧——资源轨的 `flutter assemble` 与 kernel 的 `frontend_server` 是两个 Dart 编译进程，都可能摸工程 `.dart_tool/`。因此资源轨带 `gateOnStageName: kernel` 的**保守门控**：等 kernel 结束再启动，但仍与 AOT（Wine 进程）、CMake（Ninja 进程）重叠。

门控的异常安全：每个阶段名对应一个 `Completer`；lane 启动前 `await gates[name].future`。阶段结束时——**无论成功 / 跳过 / 失败**——都在 `finally` 里 `complete`。kernel 编译失败绝不会让资源轨永久挂起。

计时报告：每阶段记录 `StageTiming{lane, name, elapsed}`（纯数据类）与 `formatDuration`（`<1s → NNNms`、`<60s → N.Ns`、更长 → `Mm S.Ss`），构建结束打印 `[lane] 阶段名 → 耗时` 表与总用时；并行模式额外算出「各阶段串行之和 vs 并行实际」的节省时长与百分比，一眼看出并行收益与编译瓶颈。

第 10 章 阶段 1：源码暂存与 ephemeral 生成

对应源码：`stages/source_staging_stage.dart`、`flutter_ephemeral.dart`

正常由 `flutter build windows` 顺手完成的准备工作，在 Linux 交叉下必须自行复现，否则 `flutter/CMakeLists.txt` 会因找不到 `generated_config.cmake` 而配置失败。本阶段做四件事：

(1) 复制 windows/ 到暂存目录

```

await stageDir.delete(recursive: true); // staging 完全可再生, 先清残留
await copyTree(ctx.project.windowsDir, ctx.windowsStageDir);

```

必须先删后建：否则重跑时 `copyTree` 会在已存在的插件符号链接上撞 `EEXIST(errno 17)`，失败后永远无法通过本阶段。`copyTree` **不跟随符号链接**并保留链接结构——因为插件链接常指向包根，若跟随会把示例工程的 `build/` 再扫进来形成无限嵌套（见第 21 章 `fs_utils`）。

(2) 生成 `flutter/ephemeral/`

把嵌入器文件铺进 ephemeral，供 `flutter/CMakeLists.txt` 使用：

- `flutter_windows.dll` + `flutter_windows.dll.lib` (导入库);
- 嵌入头文件: `kEmbedderHeaders` 已知清单 $\cup$ `embedderDir` 下实际存在的其余顶层 `.h` (取并集, 对 Flutter 未来新增头前向兼容);
- `cpp_client_wrapper/` (递归);
- `icudtl.dat`;
- 重新 `materializePluginSymlinks` (不依赖源工程已有的链接, 避免坏链/循环);
- 写 `generated_config.cmake` (`FLUTTER_ROOT / PROJECT_DIR / FLUTTER_VERSION*`, 格式对齐 `flutter_tools`);
- 中和 `flutter/CMakeLists.txt` 的 `flutter_assemble` (决策 2);
- `patch generated_plugins.cmake`: 为每个插件目标加 `set_target_properties(${plugin}_plugin PROPERTIES PREFIX "" IMPORT_PREFIX "")`。

最后一条为什么要做? MinGW 默认给 shared library 加 `lib` 前缀 (`libfoo.dll`), 而 Windows 运行时期望 MSVC 命名 (`foo.dll`)。 `PREFIX ""` 去 DLL 前缀, `IMPORT_PREFIX ""` 去导入库 (`.dll.a`) 前缀——否则 `dlltool` 从导入库名推导依赖 DLL 时会加 `lib` 前缀, 导致 EXE 运行时找 `libfoo.dll` 而实际产物是 `foo.dll`。

(3) 插件源码补丁与 `.rc` 归一化

- `PluginSourcePatcher().apply(...)`: 对已知有 Clang/MinGW 硬错误的插件打补丁 (详见第 16 章)。
- `_normalizeResourceScripts`: 把 `.rc` 里的转义反斜杠 `\\` 换成 `/`。 `llvm-rc` 在 Linux 上不把 `\` 当路径分隔符, `resources\\app_icon.ico` 会找不到图标; `/` 在两端都可用。

(4) 可选调试注入

`--debug-console` 时调 `instrumentRunnerMain` 给暂存的 `runner/main.cpp` 注入日志 (第 18 章)。

第 11 章 阶段 2: MSVC 编译标志翻译

对应源码: `stages/translate_flags_stage.dart`、`msvc_flag_translator.dart`

阶段 2 只是把 `MsvcFlagTranslator().transformTree(ctx.windowsStageDir)` 接入流水线; 真正逻辑全在翻译器里, 它是纯文本转换 (不涉及真实 CMake 调用), 因此可脱离构建单测。

声明式标志映射表

```
static const Map<String, String> _flagMap = {
  'EHsc': '', 'EHa': '', 'EHS': '',          // GCC 默认启用异常 → 直接移除
  'GR-': '-fno-rtti', 'GR': '-frtti',
  'std:c++17': '-std=c++17', 'std:c++20': '-std=c++20',
  'W3': '-Wall', 'W4': '-Wall -Wextra', 'WX': '-Werror', 'WX-': '-Wno-error',
  'utf-8': '-finput-charset=UTF-8 -fexec-charset=UTF-8',
  // ...
};
```

用 `_boundaryRegExp` 生成带边界的正则, 避免 `/W3` 误配进 `/W3X`、`/GR` 命中 `/GR-` 前缀、`/WX` 误配 `/WX-`。

`transformContent` 的四步执行顺序 (有意为之)

1. `_neutralizeHasExceptions`: 把无条件的 `"_HAS_EXCEPTIONS=0"` 改写为 `$_<$<CXX_COMPILER_ID:MSVC>:_HAS_EXCEPTIONS=0>`。该宏会破坏 MinGW 的 `libstdc++/libc++` 头; 包裹后 MSVC 仍定义、Clang 侧变 no-op。先做是因为此时尚未注入 `APPLY_STANDARD_SETTINGS`, 不会误伤注入块 MSVC 分支里的该宏。
2. `_transformApplyStandardSettings`: 把 `function(APPLY_STANDARD_SETTINGS ...)` 改写为 `if(MSVC)...else()...endif()` 双分支——MinGW 侧走 `-Wall -Werror` (不改变交叉结果), 保留 MSVC 分支

使改写后的 CMake 在真实 Windows 上仍可原样构建。整段用保护标记 `_guardBegin/_guardEnd` 包裹。

3. `_translateFlags`：按行翻译其余标志；被保护标记包裹的行整体跳过（里面的 `/W4 /WX` 不能被翻掉）。支持多行调用：进入 `target_compile_options(` 等未闭合括号区后，续行同样按旗标行处理。未识别的 `/X` 不静默丢弃而是记 warning，避免“悄悄漏译”导致隐性行为差异；`/wdNNNN`（按编号抑制告警）在 clang 无等价物、会被当输入文件报错，直接移除。
4. `_translateLibRefs`：`"foo.lib"` → `"foo"`，让 CMake 用 `-l` 去找库。

`_detectUnknownFlags` 会排除路径片段（后紧跟 `/`，如 `/usr/include`）和已知可忽略 token（`/D`、`/I`、`/Fo` 等），尽量减少误报。这一章是“不碰源码只改文本”红线的典型体现。

第 12 章 阶段 3：Dart Kernel 编译

对应源码：`stages/compile_kernel_stage.dart`

用 `frontend_server` 把 Dart 入口编译成 kernel `app.dill`，参数对齐 `flutter_tools` 的 `KernelSnapshot` 目标。

纯 Dart 插件注册器不能漏

`path_provider_windows` / `shared_preferences_windows` 等纯 Dart 插件的 `registerWith()` 由 `dart_plugin_registrant.dart` 调用。必须像 `flutter_tools` 那样把它作为 `--source` 传给 `frontend_server`，并用 `-Dflutter.dart_plugin_registrant` 指定 URI，否则运行时 `MissingPluginException`（如 `getApplicationDocumentsDirectory` 无实现）。

核心参数

```
final args = [
  ctx.env.frontendServerSnapshot,
  '--sdk-root', ctx.env.patchedSdkPath(product: ctx.mode.isProduct),
  '--target=flutter',
  for (final d in ctx.mode.kernelModeDefines) '--define=$d', // kReleaseMode 等
  if (!ctx.mode.isAot) '--enable-asserts', // debug 走 JIT
  if (ctx.mode.isAot) ...['--aot', '--tfa'], // AOT 整程序转换 + TFA
  for (final d in ctx.dartDefines) '--define=$d', // 用户 --dart-define
  '--packages', ctx.project.packageConfig,
  if (hasRegistrant) ...['--source', registrant.path, /* ... */],
  '--depfile', depfile, // 记录源码依赖，供增量
  '--output-dill', ctx.kernelDill,
  ctx.project.entryPoint,
];
await runner.run(ctx.env.frontendServerRuntime, args, tag: 'frontend_server');
// 用 env 自动选对的运行时（AOT 快照-dartaotruntime），避免 exit 255
```

注意现代引擎用的是标准 `frontend_server` AOT 快照：输出用 `--output-dill`（非旧版 `-o`），且不认 `--tree-shake-icons`——图标 tree-shaking 属资源打包阶段独立步骤，本阶段仅记一条 debug 提示。

`kernelModeDefines` 决定 Dart 侧 `kReleaseMode/kProfileMode/kDebugMode` 常量：release 是 `product=true`，profile 是 `profile=true`，debug 两者皆 false。

第 13 章 阶段 4：AOT 编译（Wine + gen_snapshot）

对应源码：`stages/aot_compile_stage.dart`、`wine_wrapper.dart`

仅 AOT 模式运行 (`shouldRun => ctx.mode.isAot`)。用 Wine 跑 Windows 版 `gen_snapshot.exe`，把 kernel 编成 ELF `app.so`。

WineWrapper: 集中生成包装脚本

与其每处手动拼 Wine 环境变量，这里生成一个 bash 包装脚本统一暴露运行环境：

```
#!/usr/bin/env bash
# Generated by flutter_build. Do not edit.
export WINEPREFIX="<buildRoot>/.wineprefix" # prefix 落在构建根下，避免污染用户 HOME
export WINEDEBUG=-all # 关掉 Wine 自身的冗长日志
exec "<wine>" "$@"
```

调用 gen_snapshot

```
final args = [
  '--snapshot-kind=app-aot-elf',
  '--elf=${ctx.appAotElf}',
  if (ctx.enableObfuscation) '--obfuscate',
  if (ctx.splitDebugInfoDir != null) '--split-debug-info=${ctx.splitDebugInfoDir}',
  for (final d in ctx.dartDefines) '--define=$d',
  ctx.kernelDill,
];
await runner.run(wine.scriptPath, [ctx.artifacts.genSnapshotExe(ctx.mode), ...args],
  tag: 'gen_snapshot', environment: wine.environment());
```

SDK 升级错配的自动修复

这是全项目最“聪明”的一处容错。捕获 `SubprocessException`，正则匹配 `Invalid kernel binary format version (expected (\d+), found (\d+))`：

```
if (mismatch == null) rethrow;
// gen_snapshot.exe (Windows 引擎产物) 与 frontend_server (host dart-sdk) 版本错配：
// SDK 自动升级只刷新了后者，stamp 污染使普通 precache 静默无效 → 必须 --force 强刷。
await EngineArtifactsProvisioner(...).ensure(force: true);
await _runGenSnapshot(ctx, wine); // 刷新成功后重试一次
```

stamp 指纹（增量用）放在**最后重算**，因为重试路径下 `gen_snapshot.exe` 已被替换，指纹须按新文件的 mtime+size 计算——`gen_snapshot.exe` 会被原地替换（路径不变），故指纹特意包含其 mtime+size 以便增量正确失效。

第 14 章 阶段 5: CMake 配置与构建

对应源码：`stages/cmake_build_stage.dart`、`host_env.dart`

这是交叉编译关键标志最密集的地方。所有 `-D` 都是“在 Linux 上用 clang++ 面向 Windows 目标构建 Flutter runner + 插件”所必需，改动需谨慎。

配置参数逐项拆解

参数	作用
<code>-G Ninja</code> + <code>CMAKE_MAKE_PROGRAM=ninja</code>	用 Ninja 生成器
<code>-DCMAKE_SYSTEM_NAME=Windows</code> <code>-D..._PROCESSOR=AMD64</code>	告知这是面向 Windows 的交叉构建，否则按本机 ELF 处理会套 RPATH 逻辑报错、用错 .exe/.dll 命名规则
<code>-DCMAKE_TRY_COMPILE_TARGET_TYPE=STATIC_LIBRARY</code>	编译器检测只编译静态库不链接 exe，否则因下面加了 <code>-municode</code> 而检测程序只有 <code>main</code> 会报 <code>undefined symbol: wWinMain</code>
<code>CMAKE_C/CXX_COMPILER</code> = clang/clang++	交叉编译器
<code>CMAKE_RC_COMPILER=llvm-rc</code> + <code>CMAKE_RC_FLAGS=-I <sysroot>/include</code>	资源编译器无 sysroot，需显式 -I 才找到 winres.h
<code>CMAKE_EXE_LINKER_FLAGS=-municode -static -ldwmapi -L <compatDir></code>	见下
<code>CMAKE_SHARED/MODULE_LINKER_FLAGS=-static ...</code>	DLL/模块静态链接
<code>CMAKE_CXX_FLAGS=-I <compatDir> -Wno-... -fms-extensions -include ...</code>	见下

EXE 链接标志四重作用：① 覆盖宿主（Flutter snap）经 env.sh 注入的 `-lepoxy/-lfontconfig` 等 Linux 库；② `-municode` 选宽字符入口 CRT 匹配 runner 的 `wWinMain`（否则 mingw crtexewin 引窄字符 `WinMain` 报 undefined）；③ `-static` 静态链接 libc++/libunwind 使产物自包含；④ `-ldwmapi` 兜底（部分插件用 MSVC 专属 `#pragma comment(lib, "dwmapi.lib")`），`-L compatDir` 提供 `libGdi32.a+libgdi32.a` 大小写修正软链。

CXX_FLAGS 的兼容招式（全在 flutter_build 侧解决，不改插件源码）：

- `-I compatDir`：MinGW 缺失的 Windows SDK 头垫片（第 16 章）。
- `-Wno-error=unknown-pragmas / unused-const-variable / unused-local-typedef / microsoft-extra-qualification`：这些警告 MSVC 不诊断但 Clang -Wall + -Werror 会升级为错误。`-Wno-error=X` 不受顺序影响，仅降级为 warning。`-fms-extensions` 让 Clang 识别 MSVC 扩展语法并把 extra-qualification 降为 ExtWarn（诊断组名变为 `microsoft-extra-qualification`）。
- `-include cmath -include iterator -include algorithm ...`：LLVM 23+ libc++ 收紧了标准库头的传递包含（`<algorithm>` 不再带 `<iterator>` 等），老插件普遍依赖旧可见性（如用 `round` 没 include `<cmath>`），对每个 TU 预注入常用标准头恢复旧语义。MSVC STL 未收紧，真实 Windows 构建不受影响。

净化宿主环境：`sanitizedCrossBuildEnv`

snap 版 Flutter 会导出 CFLAGS/CXXFLAGS/LDFLAGS 等（为其自带 GCC 构建 Linux 应用），CMake 会从中初始化默认标志，导致 `-lepoxy` 等漏进面向 Windows 的交叉链接（`lld: error: unable to find library -lepoxy`）。因此从环境里剥离这批变量，并配合 `includeParentEnvironment: false` 确保不再从父进程合并回来。PATH/HOME 等保留以便 cmake/ninja 运行。

`_ensureCleanCrossCache`：只在干净配置时切交叉模式

`CMAKE_SYSTEM_NAME` 只在干净配置时生效。用两个信号判定“健康的 Windows 交叉缓存”：CMakeCache.txt 里 `CMAKE_SYSTEM_NAME=Windows` 且 `build.ninja` 存在（只有 configure+generate 全成功才生成）。不满足则删 CMakeCache.txt 与 CMakeFiles/ 强制干净重配，满足则保留以维持增量。

第 15 章 阶段 6 资源打包与阶段 7 产物组装

对应源码：`stages/flutter_assets_stage.dart`、`stages/assemble_bundle_stage.dart`、`native_dll.dart`

阶段 6：资源打包（FlutterAssetsStage）

资源打包此前塞在组装阶段串行执行，等于把这段纯 host Dart 任务叠在关键路径末尾。它只依赖 Dart 工程与 Flutter SDK——既不依赖 CMake 原生轨的产物（exe / 插件 DLL），也不需要任何 Windows 二进制——因此独立成阶段，作为一条 lane 藏进 CMake 构建的时间窗口（保守门控等 kernel 完成后再启动，见第 9 章）。

复用官方逻辑（copy_flutter_bundle）

资源打包不自己实现，而是调 `flutter assemble copy_flutter_bundle`——该 target 只依赖 KernelSnapshot（不触发 gen_snapshot、无需 Windows 二进制），因此能在 Linux 上产出 flutter_assets：

```
final env = {'PROGRAMFILES(X86)': ''}; // 绕过 flutter 探测 VS 路径（Linux 上该变量不存在会导致报错）
await runner.run('<flutter>/bin/flutter', [
  'assemble', '-dTargetPlatform=windows-x64', '-dBuildMode=${ctx.mode.cliName}',
  '-dTreeShakeIcons=${ctx.treeShakeIcons}', '--output=${ctx.flutterAssetsDir}',
  'copy_flutter_bundle',
], workingDirectory: ctx.project.root, environment: env, stream: true);
```

生成后若 flutter_assets 仍为空，明确 warn（应用很可能无窗口/静默退出），提示用 `--debug-console` 排查。

阶段 7：产物组装（AssembleBundleStage）

把产物组装到 `outputDir/`，布局与官方一致。

单遍扫描 cmake_build

`_scanCmakeBuild` 一次递归遍历同时定位 runner exe 与收集所有插件 `.dll`（历史上是两次独立遍历）。exe 偏好顺序：`runner/` 子目录 → 根目录 → 任意候选，并跳过 `CMakeFiles/` 里的编译器探测产物。插件 DLL 因阶段 1 已设 `PREFIX ""`，此处直接按原始文件名拷贝。

按模式选引擎 DLL + icu + app.so

```
final engineDll = ctx.artifacts.flutterWindowsDllForMode(ctx.mode); // 决策/第7章
// 拷 flutter_windows.dll、data/icudt1.dat; AOT 且存在则拷 data/app.so
```

预构建原生 DLL 的两步补齐（NativeDllScanner）

有些插件依赖预编译 Windows DLL（如 `opencv_world490.dll`），在 CMakeLists 里以 Windows 绝对路径或预编译产物路径引用，且常被 `if(EXISTS ...)` 包裹——Linux 上文件不存在就被静默跳过，最终只在运行时暴露为 `DynamicLibrary.open` 失败。补两步：

1. 按声明精确解析（`copyResolvedReferencedDlls`）：从插件 `windows/CMakeLists.txt` 提取 `.dll` 字面引用（`referencedDllPaths` 剔除 `#` 注释、忽略 `$<TARGET_FILE:...>` 生成目标），把 `${CMAKE_CURRENT_SOURCE_DIR}/${CMAKE_CURRENT_LIST_DIR}` 或纯相对引用解析成绝对路径并拷贝——能覆盖位于 `.pub-cache` 内、广度扫描会跳过的插件。Windows 绝对路径（`C:/...`）无法解析，交下一步。
2. 广度扫描兜底（`copyPrebuiltDlls`）：在 `dllSearchRoot`（默认项目根祖父目录）下递归搜 `.dll`（深度上限 5，跳过 `build/` `.pub-cache/` `.git/` `snap` 等），按小写基名去重后拷入。

最后 `verifyPluginNativeDlls` 编译期校验：插件声明要打包却缺失的 DLL，集中告警并说明缺失原因（"硬编码 Windows 路径"vs"Windows 预编译产物未在 Linux 侧生成"），而非留到运行时。

第四部分 · 兼容性工程

本部分把散在各阶段的"兼容性招数"归纳成三个专题：编译期垫片/补丁、增量构建、调试仪表。它们是本项目从"能编译"走向"能真用"的精华。

第 16 章 MinGW 兼容垫片与插件源码补丁

对应源码：`mingw_compat.dart`、`plugin_source_patcher.dart`

能用垫片解决就不改源码

`mingw_compat.dart` 用一张 `kMingwCompatHeaders` 表为 MinGW-w64 缺失的 Windows SDK 头生成垫片头文件，写进 `mingwCompatDir` 并由 CMake 的 `-I` 加入搜索路径。典型几例：

被引用的头	问题	垫片做法
<code>shobjidl_core.h</code>	Win10 SDK 拆出的头，MinGW 只有 <code>shobjidl.h</code>	<code>#include <shobjidl.h></code>
<code>Windows.h</code>	Linux 大小写敏感，MinGW 发的是小写 <code>windows.h</code>	<code>#include <windows.h></code>
<code>VersionHelpers.h</code>	同上，MinGW 提供小写 <code>versionhelpers.h</code>	<code>#include <versionhelpers.h></code>
<code>sal.h</code>	MSVC SAL2 注解（ <code>_Frees_ptr_opt_</code> ）MinGW 没有	<code>include_next <sal.h></code> 后把注解空展开
<code>codecvt</code>	LLVM 23+ libc++ 不再传递暴露 <code>std::wstring_convert</code>	<code>include_next <codecvt></code> 后补 <code>#include <locale></code>

`include_next` 是 clang/gcc 特性，先放行真系统头再补齐缺失——MSVC 永远见不到这些垫片（只在交叉构建时经 `-I` 注入），因此不影响真实 Windows 构建。`materializeMingwCompat` 只在内容变化时写入，避免时间戳变化触发 ninja 全量重编。

另外还创建库大小写修正软链 `libGdi32.a → libgdi32.a`：`-fms-extensions` 处理 `#pragma comment(lib, "Gdi32.lib")` 会传 `-lGdi32`，但 MinGW 库文件全小写，Linux 大小写敏感找不到，用软链桥接。

源码补丁是最后手段

`plugin_source_patcher.dart` 目前仅保留极少数无法用标志解决的 C++ 类型硬错误。设计要点：需要补丁的插件目录会从符号链接替换为真实副本（`copyTree`），绝不修改 pub-cache 原件。例如：

- `hotkey_manager_windows`：`EncodableMap({{"identifier", identifier}})` 在 Clang/libc++ 下无法把 `{...}` 推导为 `pair<EncodableValue, EncodableValue>`（不允许两次用户定义转换），显式包 `EncodableValue` 后仍与 MSVC 兼容。
- `file_selector_windows`：`IFileDialogPtr dialog_ = nullptr;` 在 MinGW comip.h 下二义（三个指针构造隐式转换同级别）→ 改为默认构造；并删除重复的 `_COM_SMARTPTR_TYPEDEF` 调用（MinGW 展开含 inline 函数定义会 redefinition）。

两例补丁后都保持 MSVC 可原样构建——完美体现决策 6。

第 17 章 增量构建机制

对应源码：`incremental.dart`

基于 mtime 新鲜度与输入指纹（stamp）判断能否跳过昂贵重编（kernel / AOT）。采用经典 make 风格：

```
bool isUpToDate({required outputPath, required inputPaths, stampPath, expectedStamp}) {
  if (!File(outputPath).existsSync()) return false;           // 产物不存在 → 重编
  if (stampPath != null) {
    if (!File(stampPath).existsSync()) return false;           // 无指纹 → 重编
    if (File(stampPath).readAsStringSync() != expectedStamp) return false; // 指纹不符
  }
  final outMtime = File(outputPath).lastModifiedSync();
  for (final input in inputPaths) {
    if (!File(input).existsSync()) return false;               // 依赖缺失 → 重编
    if (File(input).lastModifiedSync().isAfter(outMtime)) return false; // 依赖更新 → 重编
  }
  return true;                                                  // 全部满足 → 可跳过
}
```

任何不确定都保守地判为"需要重编", 绝不冒险产出陈旧产物。 `--no-incremental` 完全关闭。

depfile 解析: frontend_server 的 `--depfile` 是 Makefile 风格 `<output>: <dep1> <dep2> ...`, `parseDepfileInputs` 消解行续接 (`\` + 换行→空格) 并还原被转义的空格/反斜杠, 取第一个 `:` 后的依赖列表。

stamp 指纹: `hashInputs` 对一组字符串片段算 sha256, 把**不体现在依赖文件里**的输入纳入判断。kernel 阶段指纹含: `mode/product/aot`、`kernelModeDefines`、`dartDefines`、入口、`sdkRoot`、`engineHash=${ctx.env.engineCommitHash}` (SDK 升级后 `patched_sdk/frontend_server` 都变了但 depfile 只记源码, 不纳入会误命中)、注册器是否存在。AOT 阶段指纹含: `混淆/split-debug/dartDefines/gen_snapshot` 路径及其 `mtime+size` (因它会被原地替换)。

第 18 章 调试仪表: 把引擎日志接回控制台

对应源码: `debug_instrumentation.dart`

背景痛点: Flutter runner 是 GUI 子系统程序 (`-mwindows`)。标准 `main.cpp` 只在"检测到调试器"时才建控制台, 且从 PowerShell 启动时虽 `AttachConsole` 到父控制台, 却**没把 stdout/stderr 重开到 CONOUT\$**, 引擎 `stderr` 日志根本出不来; 启动失败还直接 `return EXIT_FAILURE` 静默退出——这就是"运行后无窗口、无任何提示"的元凶。

`--debug-console` 时对**暂存副本** `runner/main.cpp` 做四处最小改写 (幂等, 以 `// flutter_build debug instrumentation` 作哨兵):

1. 补 `#include <stdio.h>` (`freopen_s/fprintf`) 与 `<flutter_windows.h>` (`ResyncOutputStreams`)。
2. **始终**附着到启动它的控制台, 没有则 `AllocConsole()` 新建; 随后 `freopen_s(..., "CONOUT$", "w", stdout/stderr)` + `FlutterDesktopResyncOutputStreams()`, 使引擎日志实时显示在该控制台。
3. 启动失败处向 `stderr` 输出诊断 ("likely empty data/flutter_assets or missing data/app.so")。
4. 正常退出点先 `fflush` 再 `FreeConsole()`, 使 PowerShell/cmd 在关闭程序后干净回到提示符。

默认关闭 (产出干净 GUI 程序, 双击不弹控制台); 只在排查静默失败时开启。

第五部分 · CLI、部署与基础设施

第 19 章 命令详解: doctor / precache / windows / clean

对应源码: `bin/flutter_build.dart` 与 `lib/src/commands/`

bin 入口用 `args` 包的 `CommandRunner` 组装四个子命令, 顶层标志: `-v/--verbose`、`--no-color`、`--cache-dir`、`--version`。`main()` 统一 catch: `UsageException` → exit 64; `ToolException` → 按分层 exitCode 渲染 (不输出 stack trace); 其他 → exit 1。

doctor: 只读诊断, 不改任何状态

分组输出五个检查项: Flutter SDK / 目标工程 / 交叉工具链 / 引擎产物 / 本地缓存。亮点是 `_scanForRiskyPluginCode`: 对每个有原生代码的插件递归扫描 `.cpp/.h`, 匹配 `winrt::`、`<winrt/`、`#include <d3d12`、`__uuidof` 四类 MinGW 不友好模式, 在撞编译器错误前就告警。允许在非工程目录跑 (此时只检查 host)。`--allow-download` 才会在 doctor 期间下载工具链。

precache: 为离线/CI 预热

一次性下载 LLVM-MinGW 与 Windows 引擎产物。`--toolchain-only` / `--engine-only` 各取一端, `--toolchain-path` 直接指定已有安装。

windows: 完整构建 (主命令)

`run()` 依次: `_pickFlavor()` (三标志互斥, 默认 release) → 加载 project/env/paths → provision toolchain (`--no-precache` 时 `allowDownload=false`) → ensure engine artifacts → 组装 `BuildContext` → `BuildPipeline().run()` → `_maybeDeploy()`。`--obfuscate` 必须伴 `--split-debug-info` 否则报错。并行由 `--[no-]parallel` 控制 (默认三 lane 并行, `--no-parallel` 回退串行); 每次构建结束打印逐阶段计时报告。完整标志表见附录 A。

clean: 删项目级输出

删 `build/win_cross/`。`--cmake` 只删各模式的 `cmake_build/` (保留 intermediates 与产物) ——改了编译标志/垫片后想强制 CMake 重配又不重跑昂贵 AOT 时用。`-o` 指定要清理的根。绝不动工具链/引擎缓存。

第 20 章 远程部署: config.yaml 与 scp

对应源码: `deploy.dart`

构建成功后可把产物 bundle 自动 scp 到远程 Windows 机器测试。(为何用 scp 而不是 git lfs: lfs 是把大文件以指针存进仓库, 无法"把文件通过 SSH 拷到远程 Windows 的 C 盘"; scp 能正常处理 45MB 的 flutter_windows.dll。)

DeployConfig 与查找顺序

`DeployConfig.find(startDir)` 逐级向上找 `config.yaml`, 三层优先: 项目本地 → flutter_build 工具自身目录 (全局激活时 script 指向源码 bin/) → `~/flutter_build/config.yaml`。必填 `host` (或 `ip`) 与 `remote_dir`; `password` 为空则用 SSH 密钥。模板见仓库根 `config.example.yaml`:

```
host: 192.168.1.100
username: ubuntu
password: secret      # 留空则用 SSH 密钥
auto_copy: true
remote_dir: C:/flutter_build  # 统一转正斜杠
```

远程目标 = `remote_dir/<basename>`（扁平结构，不镜像本地完整路径）。是否拷贝：显式 `--copy` / `--no-copy` 优先，否则用 `auto_copy`。

SshDeployer 执行流程

1. 先确保远程父目录存在：`ssh ... powershell -NoProfile -Command "New-Item -ItemType Directory -Force -Path '<dir>'"`（用 `-Force` 建多级、已存在不报错；不接 `| Out-Null` 管道，避免远程默认 shell 是 cmd.exe 时把 `|` 当自身管道出错）。
2. `scp -r <localDir> user@host:<remoteParent>`。

安全细节：密码认证用 `sshpass -e` + 环境变量 `SSHPASS`，而非 `-p <密码>`——避免密码出现在进程参数列表与 verbose 日志中。统一加 `-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR`；注意 scp 用大写 `-P` 指定端口。输出用容错 UTF-8 解码（远程可能 GBK 代码页，allowMalformed 避免非法字节崩溃）。

第 21 章 基础设施：日志 / 进程 / 异常 / 文件系统

对应源码：`logger.dart`、`process_runner.dart`、`exceptions.dart`、`io/fs_utils.dart`

Logger

零依赖的结构化彩色日志（便于日后换成 `package:logging` 或 CI 的 JSON 日志而不碰其他文件）。方法：`debug`（仅 verbose，走 stderr）、`info`、`step`（▶ 加粗青色）、`success`（√ 绿）、`warn`（! 黄）、`error`（X 红）、`hint`（→ 蓝）、`kv`（列对齐键值表）、`group`（标题下包裹一组日志并计时）。`Logger.instance` 是全局单例，各阶段未注入时回退到它。

ProcessRunner

包装 `Process.start`，三个恒定行为：① 实时流式 stdout/stderr 到 Logger（带 `[tag]` 前缀）；② 收集输出供失败事后诊断；③ 非零退出抛类型化的 `SubprocessException`。`which` 用 POSIX `which` 查 PATH。UTF-8 解码统一用 `allowMalformed: true`。`run` 支持 `workingDirectory`、`environment`、`includeParentEnvironment`（交叉构建时置 false 以完全控制环境）、`checked`、`stream`。

异常层次

所有异常继承 `ToolException`，携 `message` / `hint`（修复建议） / `exitCode`。退出码分层约定：1 通用、2 缺工具（wine/cmake/ninja）、3 Flutter SDK、4 工程（无 pubspec/windows 脚手架）、5 子进程失败、6 制品完整性。`main()` 统一渲染，不输出 stack trace。

fs_utils

集中目录/文件复制与体积统计。`copyTree` 不跟随符号链接并保留链接（解析为真实绝对目标重建），采用有界并发池（`kDefaultCopyConcurrency=8`）：先同步建好目录树并收集文件/链接操作，再并发执行，兼顾吞吐与文件描述符。`copyFileIfExists` 源不存在静默跳过；`dirSize` 递归统计字节数。

附录

附录 A 命令与标志速查表

```
flutter_build doctor [--allow-download]
flutter_build precache [--toolchain-only | --engine-only] [--toolchain-path DIR]
flutter_build windows [flags]
flutter_build clean [-o path] [--cmake]
```

windows 标志:

标志	用途
<code>--debug</code> / <code>--profile</code> / <code>--release</code>	构建模式（默认 release，三者互斥）
<code>-D key=value</code>	Dart <code>--define</code> ，可重复
<code>-t lib/foo.dart</code>	入口（默认 lib/main.dart）
<code>-o path</code>	输出根（默认 <code><project>/build/win_cross</code> ）
<code>--obfuscate</code> <code>--split-debug-info=<dir></code>	AOT 混淆，必须带 split-debug 目录
<code>--no-precache</code>	缺工具链/产物时报错而非自动下载
<code>--toolchain-path <dir></code>	用预装 LLVM-MinGW（同 <code>LLVM_MINGW_ROOT</code> ）
<code>--[no-]tree-shake-icons</code>	图标字体 tree-shake（默认开，尚未真正生效）
<code>--copy</code> / <code>--no-copy</code>	覆盖 config 的 auto_copy
<code>--config <path></code>	指定 config.yaml
<code>--debug-console</code>	给 runner 注入日志（排查静默退出）
<code>--[no-]incremental</code>	输入未变时跳过重编（默认开）
<code>--dll-search-root <dir></code>	预构建 DLL 搜索根（默认祖父目录）
<code>--[no-]parallel</code>	三 lane 并行（原生/Dart/资源，保守门控），默认开

顶层标志: `-v/--verbose`、`--no-color`、`--cache-dir <dir>`、`--version`。

附录 B 环境变量速查表

变量	作用
<code>LLVM_MINGW_ROOT</code>	指向预装 LLVM-MinGW 目录（跳下载）
<code>FLUTTER_BUILD_MIRROR</code>	llvm-mingw 下载镜像基址
<code>FLUTTER_BUILD_CACHE</code>	工具缓存根（优先于 <code>~/flutter_build</code> ）
<code>XDG_CACHE_HOME</code>	缓存根备选（取 <code>\$XDG_CACHE_HOME/flutter_build</code> ）
<code>FLUTTER_STORAGE_BASE_URL</code>	引擎产物下载基址（被 env 尊重）
<code>FLUTTER_VERSION</code>	无顶层 version 文件时的版本回退值
<code>SSHPASS</code>	部署密码登录时由 sshpass -e 使用

附录 C 故障排查清单

- `flutter_build: command not found`: `~/pub-cache/bin` 不在 PATH，加 `export PATH="$PATH:$HOME/.pub-cache/bin"`；或在仓库内用 `dart run flutter_build <cmd>`。

- `Neither LLVM-MinGW nor system GCC-MinGW could be found.`：选一种工具链方式（`export LLVM_MINGW_ROOT / precache / apt 装 gcc-mingw-w64-x86-64`）。
- 插件报 `<winrt/...>` 或 `<d3d12*>` 头错误：mingw-w64 的 WinRT/DX12 头不完整；`doctor` 会预先标记，可换纯 Dart 替代、本地补丁、或在真实 Windows 上单独构建该插件后把 DLL 拷到 exe 旁。
- AOT 在 Wine 下失败（`err:module:import_dll / 缺 DLL`）：确保可用的 64 位 Wine（`wine64 --version ≥ 6.0`）；精简 Ubuntu 镜像上 `sudo apt install wine64 winbind`。
- `Invalid kernel binary format version`：SDK 升级后 Windows 引擎产物未刷新。阶段 4 会自动 `--force` 重试；也可手动 `flutter precache --no-android --no-ios --windows --force`。
- `lld: error: unable to find library -lepoxy`：宿主（snap）污染了 LDFLAGS；工具已用 `sanitizedCrossBuildEnv` 剥离，若自写脚本需避免直接继承宿主环境。
- 运行后无窗口/静默退出：用 `--debug-console` 重建，从 PowerShell/cmd 运行看引擎日志（多为 flutter_assets 为空或缺 data/app.so）。

附录 D 术语表

术语	含义
AOT / JIT	提前编译 / 即时编译；release/profile 用 AOT，debug 用 JIT
kernel (<code>.dill</code>)	Dart 中间字节码，frontend_server 产出、gen_snapshot 消费
<code>app.so</code>	AOT 机器码快照，以 ELF 容器承载（引擎内置 ELF 加载器）
frontend_server	把 Dart 源码编译为 kernel 的工具（现代为 AOT 快照，需 dartaotruntime）
gen_snapshot	把 kernel 编为目标平台 AOT 码的官方工具（本工具经 Wine 跑其 Windows 版）
LLVM-MinGW	clang+lld+mingw-w64 的 Windows 交叉工具链
Wine	Linux 上的 Windows 兼容层，用于跑 gen_snapshot.exe
embedder	引擎嵌入层（flutter_windows.dll 及其头/包装）
ephemeral	flutter 构建期生成的临时目录（含嵌入器与插件链接）
stamp	输入指纹（sha256），用于增量判断
版本接缝	代码中对 Flutter 目录布局版本敏感假设的集中点

附录 E 源码文件地图

bin/flutter_build.dart	CLI 入口, 组装 CommandRunner 与全局标志
lib/flutter_build.dart	对外导出 (BuildContext/Pipeline/Toolchain/...)
lib/src/	
└ flutter_env.dart	FlutterEnv: SDK 探测 (版本接缝)
└ project.dart	FlutterProject: 工程与插件解析
└ toolchain.dart	Toolchain + Provisioner: LLVM-MinGW 下载/探测
└ engine_artifacts.dart	EngineArtifacts: 引擎产物与新鲜度 (版本接缝)
└ cache_paths.dart	CachePaths: 工具级缓存目录
└ deploy.dart	DeployConfig + SshDeployer: 远程部署
└ exceptions.dart logger.dart process_runner.dart	基础设施
└ io/fs_utils.dart	copyTree 等文件工具
└ commands/{doctor,precache,windows,clean}_command.dart	
└ build/	
└ build_context.dart	不可变构建契约 + 路径派生
└ pipeline.dart	阶段编排: lane 执行、跨轨门控与计时报告
└ build_schedule.dart	planSchedule: 并行/顺序调度纯函数 (可单测)
└ stage_timing.dart	阶段计时记录与时长格式化
└ wine_wrapper.dart	Wine 包装脚本
└ msvc_flag_translator.dart	MSVC-Clang 标志翻译
└ mingw_compat.dart	垫片头 + 库名大小写修正
└ plugin_source_patcher.dart	极少数插件源码补丁
└ flutter_ephemeral.dart	ephemeral 生成与 flutter_assemble 中和
└ host_env.dart	交叉构建环境净化
└ incremental.dart	增量判据 (depfile + stamp)
└ native_dll.dart	预构建 DLL 发现与校验
└ debug_instrumentation.dart	runner 日志注入
└ stages/	七个阶段实现

本手册完。建议结合源码与仓库内 [README.md](#) / [README.zh.md](#) 一起阅读。掌握七阶段流水线、三 lane 并行调度与"不碰源码优先"的设计红线, 就掌握了整个项目的灵魂。